

KI/NUM — Shrnutí k zápočtu

Zdeněk Touška

2025

Obsah

I	Obyčejné diferenciální rovnice (ODR)	3
1	Eulerova metoda	4
2	Heunova metoda	5
3	Runge--Kuttova metoda (RK4)	7
II	Numerická integrace	9
4	Obdélníkové pravidlo	9
5	Lichoběžníkové pravidlo	10
6	Simpsonovo pravidlo	10
III	Soustavy lineárních rovnic	13

7	Gaussova eliminace	13
8	Jacobiho metoda	14
9	Gauss–Seidelova metoda	15
IV Aproximace — Metoda nejmenších čtverců (MNČ)		17
10	Metoda nejmenších čtverců (MNČ)	17
V Interpolace		20
11	Lagrangeova interpolace	20
12	Lineární interpolace po částech	22
VI Hledání kořenů $f(x) = 0$		23
13	Bisekce	23
14	Regula falsi	24
15	Newtonova metoda	25
16	Prostá iterace (Fixed point)	26
17	Steffensenova metoda	27
18	Halleyho metoda	28

Část I

Obyčejné diferenciální rovnice (ODR)

Kdy to použít: Máš rovnici tvaru $y' = f(x, y)$ a počáteční podmínku $y(x_0) = y_0$. Analytické řešení neexistuje nebo je složité — konstruuješ průběh řešení krok po kroku.

Proč numericky a ne analyticky?

Analyticky bys musel integrovat a řešit diferenciální rovnici vzorcem. U složitějších funkcí to buď nejde vůbec, nebo je to extrémně pracné. Numericky říkáš: *neznám vzorec, ale dokážu se k řešení přiblížit krok po kroku.*

Intuice — poloha a rychlost

Představ si pohyb v prostoru:

- y_k je **poloha** v čase x_k ,
- $f(x_k, y_k)$ je **rychlost** (směrnice tečny) — říká jak rychle a jakým směrem se y mění,
- h je **časový krok** — jak daleko skočíš.

Tečna je přímka dotýkající se křivky v jednom bodě se stejným sklonem. Euler říká: pohybují se po tečně místo po skutečné křivce. Čím menší h , tím kratší úsek tečny použiješ a tím menší chyba.

Rozdíl mezi y_k a y_{k+1} je přesně to, co metoda počítá:

$$y_{k+1} - y_k = h \cdot f(x_k, y_k).$$

Nová poloha = stará poloha + krok $\times$ rychlost. Stejná logika jako $s = v \cdot t$ z fyziky.

Co znamená řád metody?

Řád říká, jak rychle roste chyba při zvětšení kroku h :

- **1. řád (Euler)** — zdvojnásobíš h , chyba se zdvojnásobí,
- **2. řád (Heun)** — zdvojnásobíš h , chyba se zčtyřnásobí,
- **4. řád (Runge–Kutta)** — zdvojnásobíš h , chyba vzroste $16\times$.

Vyšší řád = při malém h mnohem přesnější, ale citlivější na velikost kroku.

1 Eulerova metoda

Co dělá

V každém bodě se podívá na směrnici tečny a udělá malý krok tím směrem. Výsledek je lomenou čarou aproximující skutečné řešení.

Vzorec

$$y_{k+1} = y_k + h \cdot f(x_k, y_k), \quad x_{k+1} = x_k + h.$$

Proč ji použít

- Nejjednodušší implementace ze všech metod ODR.
- Vhodná jako základ nebo pro hrubý odhad.
- Stavební kámen složitějších metod (Heun, prediktor–korektor).

Kdy nestačí

- Metoda **prvního řádu** — chyba klesá lineárně s h .
- Při velkém kroku nebo rychle se měnícím řešení se chyba rychle akumuluje.
- Na dlouhém intervalu výsledek ztrácí přesnost.

Prakticky

Krok h volíš kompromisem: menší h = vyšší přesnost, ale více výpočtů.

2 Heunova metoda

Co dělá

Euler bere směrnici jen na začátku kroku — křivka se ale za ten krok zahýbe. Heun udělá **dva průchody**:

1. **Predikce** — spočítá to stejně jako Euler, dostane hrubý odhad $\tilde{y}_{k+1}$.
2. **Korekce** — spočítá směrnici i v odhadnutém bodě, vezme **průměr** obou směrnic a teprve z toho udělá skutečný krok.

y_{k+1} se nezapisuje po prvním průchodu — ten je pouze pomocný. Zapiše se až po korekci z průměru.

Vzorec

$$\tilde{y}_{k+1} = y_k + h \cdot f(x_k, y_k) \quad (\text{predikce — Eulerův krok})$$

$$y_{k+1} = y_k + \frac{h}{2} [f(x_k, y_k) + f(x_{k+1}, \tilde{y}_{k+1})]$$

(korekce — průměr směrnic)

Proč ji použít

- Metoda **druhého řádu** — výrazně přesnější než Euler.
- Cena: dvě vyhodnocení f místo jednoho.
- Jednoduchá implementace, dobrý kompromis přesnost/náročnost.

Schéma

Euler: smernice na zacatku --> krok --> $y_{\{k+1\}}$

Heun: 1) smernice na zacatku --> hruby krok --> y_{tilda}

2) smernice na konci (z hrubeho odhadu)

3) prumer --> skutecny krok --> $y_{\{k+1\}}$

3 Runge–Kuttova metoda (RK4)

Co dělá

Stejná myšlenka jako Heun, ale před tím než udělá krok, provede **4 průzkumy terénu** na různých místech uvnitř kroku. Nejde o 4 kroky dopředu — jde o 4 odhady směrnice, ze kterých pak udělá jeden skutečný krok.

Čtyři směrnice

$k_1 = f(x_k, y_k)$	začátek kroku (jako Euler)
$k_2 = f(x_k + \frac{h}{2}, y_k + \frac{h}{2} k_1)$	půlka kroku, odhad přes k_1
$k_3 = f(x_k + \frac{h}{2}, y_k + \frac{h}{2} k_2)$	půlka kroku, zpřesnění přes k_2
$k_4 = f(x_k + h, y_k + h k_3)$	konec kroku, odhad přes k_3

k_2 a k_3 jsou oba uprostřed kroku, ale vycházejí z trochu jiného odhadu polohy — k_3 je zpřesnění k_2 .

Výsledný krok

$$y_{k+1} = y_k + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

Váhy jsou 1, 2, 2, 1 — střední dva dostávají váhu 2, protože střed kroku lépe vystihuje průběh křivky než krajní body. Součet vah $1+2+2+1=6$, proto se dělí 6.

Schéma

k1 ... zacatek ----- konec

```

k2 ... zacatek ----- pulka (pres k1)
k3 ... zacatek ----- pulka (pres k2)
k4 ... zacatek ----- konec (pres k3)

```

vazeny prumer $k_1, k_2, k_3, k_4 \rightarrow y_{k+1}$

Proč ji použít

- Metoda **čtvrtého řádu** — zlatý standard v praxi.
- Cena: čtyři vyhodnocení f na jeden krok.
- Výrazně přesnější než Euler i Heun při stejném h .

Srovnání metod ODR

Metoda	Vyhodnocení f	Řád
Euler	1	1
Heun	2	2
Runge–Kutta	4	4

Část II

Numerická integrace

Kdy to použít: Máš integrál $\int_a^b f(x) dx$ a buď nemáš analytický vzorec, nebo ho nechceš počítat. Numericky interval rozdělíš na kousky a plochu pod křivkou aproximuješ jednoduchými geometrickými tvary.

Obecný princip

1. Rozděl interval $[a, b]$ na n dílků stejné šířky $h = \frac{b-a}{n}$.
2. Na každém dílku (nebo dvojici dílků) nahraď funkci jednoduchým tvarem.
3. Spočítej plochu toho tvaru.
4. Všechny plochy sečti — výsledek je aproximace integrálu.

Čím lepší tvar (parabola > přímka > obdélník), tím přesnější aproximace. Čím více dílků, tím přesnější výsledek.

4 Obdélníkové pravidlo

Co dělá

Každý dílek nahradí obdélníkem — výška je hodnota funkce v nějakém bodě dílku. Velmi hrubá aproximace, slouží především k pochopení principu.

5 Lichoběžníkové pravidlo

Co dělá

Místo obdélníku spojí hodnoty funkce na začátku a konci každého dílku **přímkou**. Plocha pod tou přímkou je lichoběžník. Respektuje že funkce se mezi dvěma body mění — přesnější než obdélník.

6 Simpsonovo pravidlo

Co dělá

Místo přímky použije **parabolu** procházející třemi body — začátek, střed a konec každé dvojice dílků. Parabola lépe vystihuje zakřivení funkce.

Podmínka

Musíš mít **sudý počet dílků**.

Proč: Simpson bere vždy **dvojici sousedních dílků** najednou — potřebuje 3 body (začátek, střed, konec té dvojice) aby mohl proložit parabolou. Dílky tedy musí jít po dvou: 2, 4, 6, 8... Kdybys měl lichý počet, poslední dílek by zbyl sám bez paraboly.

	dílek 1		dílek 2		dílek 3		dílek 4	
	_____parabola 1_____		_____parabola 2_____					
bod0		bod1		bod2		bod3		bod4

Kdy nestačí

Simpson je přesný pro **hladké funkce**. Tam kde má funkce ostré zakřivení nebo zlomy parabola nestačí funkci přesně vystihnout a chyba

roste.

Srovnání metod numerické integrace

Metoda	Tvar aproximace	Přesnost
Obdélník	konstanta	nejnižší
Lichoběžník	přímka	střední
Simpson	parabola	nejvyšší

Proč Simpson dělí 3?

Vzorec pro jednu dvojici dílků:

$$\frac{h}{3}(f_0 + 4f_1 + f_2)$$

Prostřední bod má váhu 4, krajní body váhu 1. Prostřední bod dostává větší váhu protože je uprostřed paraboly kde nejvíc určuje její tvar.

Jak se volí počet dílků n ?

- Zadání říká přímo: $n = 100$, $n = 1000$ — použij to.
- Zadání neřekne: zdvojuj n dokud se výsledek přestane měnit — to znamená dostatečnou přesnost.

Co se stane při lichém n u Simpsona?

Metoda se nedá aplikovat — implementace buď vyhodí chybu, nebo se poslední dílek dopočítá lichoběžníkem zvlášť.

Kdy použít kterou metodu?

- Zadání říká "Simpson" $\rightarrow$ Simpson, sudý n .
- Zadání říká "lichoběžník" $\rightarrow$ lichoběžník, libovolný n .
- Zadání nespecifikuje a chceš přesnost $\rightarrow$ Simpson.

Část III

Soustavy lineárních rovnic

Kdy to použít: Máš soustavu $Ax = b$ a chceš najít vektor x .

Dvě strategie řešení

- **Přímá metoda (Gaussova eliminace)** — projde jednou, vrátí přesný výsledek. Složitost $O(n^3)$ — pro velké n pomalé.
- **Iterační metody (Jacobi, Gauss-Seidel)** — začnou odhadem a postupně ho zpřesňují. Pro velké soustavy výrazně rychlejší.

Praktické pravidlo

- Matice velká + diagonálně dominantní → **Jacobi nebo Gauss-Seidel**.
- Cokoliv jiného → **Gaussova eliminace**.

7 Gaussova eliminace

Co dělá

Přímá metoda — dva kroky:

1. **Přímý chod** — postupně vynuluje prvky pod diagonálou, vznikne horní trojúhelníková matice.
2. **Zpětný chod** — od poslední rovnice zpět vypočítá všechny neznámé.

Podmínka použití

Matice musí být **regulární** — nesmí mít nulový determinant, soustava musí mít jednoznačné řešení.

Pivot

Diagonální prvek kterým se v daném kroku dělí. Pokud je nulový nebo velmi malý — metoda selže nebo je numericky nestabilní.

8 Jacobiho metoda

Co dělá

Z každé rovnice vyjádříš jednu neznámou. Začneš počátečním odhadem (třeba samé nuly) a iteruješ.

Klíčové pravidlo: v každém kole počítáš všechny nové hodnoty **výhradně ze starých** — teprve až máš celé nové kolo, přepíšeš odhad.

Příklad

Soustava:

$$2x_1 + x_2 = 5, \quad x_1 + 3x_2 = 7$$

Vyjádření neznámých:

$$x_1 = \frac{5 - x_2}{2}, \quad x_2 = \frac{7 - x_1}{3}$$

Start: $x_1 = 0$, $x_2 = 0$.

Kolo 1 (obě hodnoty ze starých dat):

$$x_1^{new} = \frac{5 - 0}{2} = 2,5 \quad x_2^{new} = \frac{7 - 0}{3} = 2,33$$

Kolo 2:

$$x_1^{new} = \frac{5 - 2,33}{2} = 1,33 \quad x_2^{new} = \frac{7 - 2,5}{3} = 1,5$$

A tak dál dokud se hodnoty přestanou měnit.

9 Gauss–Seidelova metoda

Co dělá

Stejně jako Jacobi, ale s jednou změnou: jakmile spočítáš novou hodnotu jedné neznámé, **okamžitě ji použiješ** v dalším výpočtu ve stejném kole.

Příklad — stejná soustava

Start: $x_1 = 0$, $x_2 = 0$.

Kolo 1:

$$x_1^{new} = \frac{5 - 0}{2} = 2,5$$

Okamžitě použij $x_1 = 2,5$:

$$x_2^{new} = \frac{7 - 2,5}{3} = 1,5$$

Kolo 2:

$$x_1^{new} = \frac{5 - 1,5}{2} = 1,75 \quad x_2^{new} = \frac{7 - 1,75}{3} = 1,75$$

Správné řešení je $x_1 = 1,6$, $x_2 = 1,8$ — Gauss–Seidel se přibližuje rychleji než Jacobi díky čerstvějším datům.

Proč je Gauss–Seidel rychlejší?

Pracuje s aktuálnějšími hodnotami — jakmile zpřesní jednu neznámou, ostatní z toho hned těží ve stejném kole. U Jacobiho musí všichni čekat na konec kola.

Podmínka konvergence obou metod

Matice A musí být **diagonálně dominantní** — každý prvek na diagonále musí být větší než součet ostatních prvků v tom řádku.

Příklad diagonálně dominantní matice:

$$\begin{pmatrix} 10 & 1 & 2 \\ 1 & 8 & 1 \\ 2 & 1 & 9 \end{pmatrix}$$

Řádek 1: $10 > 1 + 2 = 3$ ✓ Řádek 2: $8 > 1 + 1 = 2$ ✓ Řádek 3: $9 > 2 + 1 = 3$ ✓

Pokud podmínka není splněna — použij Gaussovu eliminaci.

Proč iterační metody pro velké soustavy?

- Gaussova eliminace: n^3 operací — pro $n = 1000$ je to 10^9 operací.
- Gauss–Seidel: $n \times$ počet kol — pro $n = 1000$ a 50 kol jen 50 000 operací.

Část IV

Aproximace — Metoda nejmenších čtverců (MNČ)

Kdy to použít: Máš naměřená data zatížená chybou nebo šumem a chceš najít funkci která vystihuje celkový trend — ne funkci která prochází každým bodem.

Interpolace vs. Aproximace

- **Interpolace** — funkce prochází přesně každým bodem, včetně chybných měření.
- **Aproximace** — funkce co nejlépe vystihuje trend, chybná měření "vyhladí".

10 Metoda nejmenších čtverců (MNČ)

Základní myšlenka

Pro každý bod spočítáš odchylku mezi naměřenou hodnotou a hodnotou tvé funkce. Odchylky umocníš na druhou a sečteš — tento součet chceš minimalizovat.

$$\min \sum_{i=1}^n (y_i - f(x_i))^2$$

Proč na druhou:

- Kladné a záporné odchylky by se jinak rušily.

- Větší chyby dostanou větší váhu než malé.

Jak se minimalizuje — normální rovnice

Chceš proložit data **přímkou** $f(x) = ax + b$. Položíš parciální derivace součtu čtverců rovno nule:

$$\frac{\partial S}{\partial a} = 0, \quad \frac{\partial S}{\partial b} = 0.$$

Vyjdou dvě rovnice o dvou neznámých a a b — **normální rovnice**:

$$\left(\sum x_i^2\right) a + \left(\sum x_i\right) b = \sum x_i y_i$$

$$\left(\sum x_i\right) a + n \cdot b = \sum y_i$$

Pro polynom stupně k vyjde soustava $(k + 1)$ rovnic o $(k + 1)$ neznámých — princip stejný.

Napojení na Gaussovu eliminaci

MNČ samo o sobě numericky nic neřeší — pouze převede problém "najdi nejlepší křivku" na soustavu lineárních rovnic $Ax = b$. Tu pak řeší Gaussova eliminace.

namerena data

|

v

MNC --> sestaveni normalnych rovnic --> matice A a vektor b

|

v

Gaussova eliminace --> resi Ax = b

|

v

koeficienty aproximacni funkce (a, b, c, ...)

Podmínka použití

Počet datových bodů musí být **větší** než počet koeficientů. Kdybys měl stejný počet, nedostaneš aproximaci ale interpolaci — funkce by prošla přesně každým bodem.

Varianty podle tvaru aproximační funkce

- **Přímka** $f(x) = ax + b$ — 2 koeficienty, nejjednodušší případ.
- **Polynom stupně k** — $k + 1$ koeficientů, normální rovnice větší.
- **Trigonometrická báze** (sinus, kosinus) — vhodná pro periodická data, princip MNČ stejný, liší se jen volbou bázevých funkcí.

Část V

Interpolace

Kdy to použít: Máš body a chceš křivku která jimi prochází **přesně**.
Typicky chceš zjistit hodnotu funkce mezi naměřenými body.

Interpolace vs. Aproximace — shrnutí

- **Interpolace** — křivka prochází přesně zadanými body.
- **Aproximace** — křivka se k bodům co nejvíce přibližuje, nemusí jimi procházet.

Co je výsledkem interpolace?

Výsledkem je **polynom** — křivka která může mít kopce, údolí, zatáčky.
Čím více bodů, tím vyššího stupně polynom:

- 2 body $\rightarrow$ stupeň 1 $\rightarrow$ přímka
- 3 body $\rightarrow$ stupeň 2 $\rightarrow$ parabola
- 4 body $\rightarrow$ stupeň 3 $\rightarrow$ křivka s jedním ohybem
- n bodů $\rightarrow$ stupeň $n - 1 \rightarrow$ stále složitější křivka

11 Lagrangeova interpolace

Co dělá

Sestaví polynom stupně $n - 1$ který prochází přesně všemi n body.
Každý bod dostane svůj **základní polynom** $L_i(x)$ který:

- v bodě x_i má hodnotu **1**,
- ve všech ostatních uzlech x_j má hodnotu **0**.

Každý L_i tedy zodpovídá pouze za svůj bod — v ostatních bodech je nula a neruší.

Vzorec

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n-1} \frac{x - x_j}{x_i - x_j} \quad (\text{součin přes všechny body kromě } i)$$

$$p(x) = \sum_{i=0}^{n-1} y_i \cdot L_i(x)$$

Rungeho jev

Při velkém počtu **rovnoměrně rozložených** uzlů polynom vysokého stupně na krajích intervalu divoce kmitá — paradoxně více bodů = horší výsledek na krajích.

Proč: polynom má velkou svobodu pohybu. Uprostřed ho body drží z obou stran. Na krajích ho drží jen z jedné strany — a tam ujede.

Čebyševovy uzly — řešení Rungeho jevu

Místo rovnoměrného rozmístění dáš body **hustěji ke krajům** a řidčeji uprostřed. Kraje jsou problematické — více bodů tam = polynom nemá prostor ujíždět.

rovnomerne:	--*--*--*--*--*--*--	
Cebysev:	*--*--*--*--*--*--*	
	huste	huste
	kraje	kraje

Vzorec pro uzly:

$$x_k = \cos \left(\frac{2k+1}{2n+2} \pi \right)$$

12 Lineární interpolace po částech

Co dělá

Místo jednoho velkého polynomu spojí vždy **dva sousední body** **přímku**. Výsledek je lomená čára.

Na každém úseku $[x_i, x_{i+1}]$:

$$p(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i} (x - x_i)$$

Výhody oproti Lagrangeovi

- Žádný Rungeho jev — nepoužívá polynom vysokého stupně.
- Stabilní pro libovolný počet bodů.
- Jednoduchá implementace.

Nevýhody

- Na spojích bodů vznikají zlomy — křivka není hladká.
- Méně přesná než Lagrange pro hladké funkce.

Část VI

Hledání kořenů $f(x) = 0$

Kdy to použít: Máš funkci a chceš najít x kde je rovna nule. Analyticky to často nejde — iteruješ.

Dvě skupiny metod

- **Intervalové** — potřebuješ interval $[a, b]$ kde funkce mění znaménko. Zaručená konvergence. (Bisekce, Regula falsi)
- **Bodové** — potřebuješ jen počáteční odhad. Rychlejší, ale konvergence není zaručena. (Newton, Sečny, Prostá iterace, Steffensen, Halley)

13 Bisekce

Co dělá

Vezme střed intervalu $c = \frac{a+b}{2}$, podívá se na $f(c)$ a podle znaménka zjistí ve které polovině kořen leží. Tu polovinu vezme jako nový interval a opakuje.

Podmínka použití

$$f(a) \cdot f(b) < 0$$

Funkce musí měnit znaménko — zaručuje existenci kořene v intervalu.

Vzorec

$$c = \frac{a + b}{2}$$

- $f(a) \cdot f(c) < 0 \rightarrow$ kořen vlevo $\rightarrow$ nový interval $[a, c]$
- jinak $\rightarrow$ nový interval $[c, b]$

Proč zaručená konvergence?

Interval se v každém kroku přesně půlí — kořen v něm vždy zůstává.
Po n krocích je interval $\frac{b-a}{2^n}$ — vždy konverguje.

Slabina

Pomalá — vždy dělí na půl bez ohledu na tvar funkce.

14 Regula falsi

Co dělá

Místo středu intervalu vezme průsečík **sečny** procházející body $(a, f(a))$ a $(b, f(b))$ s osou x — nový bod je blíže kořenu než pouhý střed.

Vzorec

$$c = \frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)}$$

Aktualizace intervalu stejná jako u bisekce.

Problém — stagnace

Jeden kraj intervalu se může dlouho nehýbat — pokud je $f(b)$ velmi malé, sečna míří pořád skoro na stejné místo a c se posouvá jen velmi pomalu.

Kdy co použít

- Funkce má hladký tvar $\rightarrow$ regula falsi bývá rychlejší.
- Jeden krajní bod blízko nuly $\rightarrow$ bisekce je spolehlivější.

15 Newtonova metoda

Co dělá

V aktuálním bodě x_k sestrojí **tečnu** ke grafu funkce. Průsečík tečny s osou x je nový odhad kořene x_{k+1} . Potřebuje derivaci $f'(x)$.

Vzorec

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Proč je rychlejší než bisekce?

Kvadratická konvergence — počet správných číslic se v každém kroku přibližně zdvojnásobí:

krok 1: $x = 1.5xxxxxx$

krok 2: $x = 1.618xxxxx$

krok 3: $x = 1.6180339x$

krok 4: $x = 1.618033988749\dots$

Kdy selže

- Špatný počáteční bod — tečna míří daleko od kořene.
- $f'(x_k) = 0$ — tečna je vodorovná, průsečík neexistuje.
- Více kořenů — může skočit k jinému kořenu než chceš.

Prakticky

Často se kombinuje: bisekce pro hrubý odhad, Newton pro rychlé zpřesnění.

16 Prostá iterace (Fixed point)

Co dělá

Rovnici $f(x) = 0$ přepíšeš do tvaru $x = g(x)$ a iteruješ:

$$x_{k+1} = g(x_k)$$

Hledáš **pevný bod** — bod kde g zobrazuje bod sama na sebe.

Podmínka konvergence

$$|g'(x)| < 1 \quad \text{v okolí kořene}$$

Pokud je křivka $g(x)$ "méně strmá" než přímka $y = x$ — iterace konverguje. Pokud je strmější — diverguje.

Hlavní slabina

Záleží na tom jak rovnici přepíšeš — jeden přepis konverguje, jiný diverguje. Například $x^2 - x - 2 = 0$ lze přepsat jako:

- $x = x^2 - 2$ — může divergovat,
- $x = \sqrt{x + 2}$ — konverguje.

17 Steffensenova metoda

Co dělá

Zrychluje prostou iteraci. Místo pomalého přibližování udělá dva kroky prosté iterace a z výsledků odhadne kde kořen skutečně je — pak tam rovnou skočí.

Postup

Dva pomocné kroky:

$$x_1 = g(x_k), \quad x_2 = g(x_1)$$

Korekce:

$$x_{k+1} = x_k - \frac{(x_1 - x_k)^2}{x_2 - 2x_1 + x_k}$$

Výsledek

Kvadratická konvergence — stejně rychlá jako Newton, ale **bez derivace**.

Kdy selže

- Jmenovatel $x_2 - 2x_1 + x_k$ je nula nebo velmi malý.
- Špatný počáteční odhad nebo nevhodná funkce $g(x)$.

Prosta iterace: $x_k \rightarrow x_{k+1} \rightarrow x_{k+2} \rightarrow x_{k+3} \rightarrow \dots$ pomalu
Steffensen: $x_k \rightarrow (\text{dva kroky}) \rightarrow \text{skok} \rightarrow x_{k+1}$ rychle

18 Halleyho metoda

Co dělá

Rozšiřuje Newtonovu metodu — místo tečny (lineární aproximace) používá **kvadratickou aproximaci** funkce. Potřebuje první i druhou derivaci $f'(x)$ a $f''(x)$.

Vzorec

$$x_{k+1} = x_k - \frac{2 f(x_k) f'(x_k)}{2 [f'(x_k)]^2 - f(x_k) f''(x_k)}$$

Konvergence

Kubická — řád 3. Počet správných číslic se v každém kroku přibližně ztrojnásobí. Rychlejší než Newton, ale za cenu druhé derivace.

Srovnání všech metod

Metoda	Potřebuje	Konvergence	Zaručená?
Bisekce	interval $[a, b]$	lineární	ano
Regula falsi	interval $[a, b]$	superlineární	ano
Newton	x_0, f'	kvadratická	ne
Prostá iterace	x_0 , přepis	lineární	ne
Steffensen	x_0 , přepis	kvadratická	ne
Halley	x_0, f', f''	kubická	ne